Algoritma Struktur dan Data. - Assignment.

Name : Zaitunisa Hartining ciyas

NIM : 24/544067 / TK 160473 - C.

※ Problem : 1

Snippet A : $A = n$, $B = \frac{n(n-1)}{2}$

Line	Operation	Executions	Cost / exec
(a).	Sum= 0 ;	1	1
(b).	Outer init : i =0 ;	1	1
(b).	Outer cond : i < n ;	A +1	1
(b).	Outer incr : i++ ;	A	1
(c).	Inner init : j =0 ;	A	1
(c).	Inner cond : j < i ;	B +A	1
(c).	Inner incr : j++ ;	B	1
(d).	Body : arr [i], arr [j]	B	2
(d).	arithmatic (+)	B	1
(d).	assignment to sum	B	1

$T(n) = 1 + 1 + (A+1) + A + A + (B+A) + B + 2B + B + B$

$= 3 + 4A + 6B.$

$T(n) = 3 + 4n + 6\left(\frac{n(n-1)}{2}\right) = 3 + 4n + 3n(n-1) = 3 + 4n + 3n^2 - 3n = 3n^2 + n + 3$

$T(n) = 3n^2 + n + 3$

Big-O class : $O(n^2)$

Verification : $(n=4) = 3(4)^2 + 4 + 3 = 48 + 4 + 3 = 55$ Ops.

Best / Worst Case : Tidak berbeda, karena jumlah iterasi tidak bergantung pada isi array.

Snippet B : $A = [\log_2 n] + 1$

Line	Operation	Executions	Cost / exec
(a).	lo =0	1	1
(a).	hi = n-1	1	2 (1 arith + 1 assign)
(a).	target = 42	1	1
(b).	found = false	1	1
(c).	While cond (lo ≤ hi)	A+1	1
(d).	hi - lo	A	1
(d).	/2	A	1
(d).	lo + (...)	A	1
(d).	assign mid	A	1
(e).	arr [mid]	A	1

GELATIK

Line	Operation	Execution	Cost /exec
(e).	Comparison ==	A	1
(f).	arr [mid]	A	1
(f).	Comparison <	A	1
(g).	mid + 1 or mid − 1	A	1
(g).	assignment (lo or hi)	A	1

$T(n) = 5 + (A+1) + 10A = 6 + 11A$, Substitusi $A = [\log_2 n] + 1$

- $T(n) = 6 + 11([\log_2 n] + 1)$
- Big-O Class = $O(\log n)$
- Best Case : $O(1)$ → langsung ketemu di tengah
- Worst case : $O(\log n)$

Snippet C : $A = n$, $B = \frac{n(n-1)}{2}$, $C = [n/2]$

$$D = B.C = \frac{n(n-1)}{2} \cdot \frac{n}{2} = \frac{n^3}{4} - \frac{n^2}{4}$$

Line	Operation	Execution	Cost /exec
(a).	Count = 0	1	1
(b).	Outer init	1	1
(b).	Outer cond	A + 1	1
(b).	Outer incr	A	1
(c).	middle init	A	2
(c).	middle cond	B + A	1
(c).	middle incr	B	1
(d).	inner init	B	1
(d).	inner cond	D + B	1
(d).	inner incr (k+ = 2)	D	2
(e).	arr [i]	D	1
(e).	arr [j]	D	1
(e).	! =	D	1
(e).	arr [k]	D	1
(e).	>	D	1
(f).	Count ++	D.	1

$T(n) = 6D = 6\left(\frac{n^3}{4} - \frac{n^2}{4}\right)$

- Best Case = Tidak berbeda (loop selalu penuh)
- Worst Case = $O(n^3)$

- $T(n) = \frac{3}{2}n^3 - \frac{3}{2}n^2 + O(n^2)$
- Big-O Class = $O(n^3)$

※Problem : 4

Consider the pushback function from Problem 2, which doubles the capacity whenever size == capacity starting from capacity = 2.

Part A - Trace.

For each call, record : the value pushed, the resulting size, the resulting capacity, and the cost of that specific call (cost = 1 for normal append ; cost = 1 + old size when a resize occurs).

Call ※	Value	Size after	Cap after	Cost
1	10	1	2	1
2	20	2	2	1
3	30	3	4	3 (1+ old size = 1+2)
4	40	4	4	1
5	50	5	8	5 (resize = 1+4)
6	60	6	8	1
7	70	7	8	1
8	80	8	8	1
9	90	9	16	9 (resize = 1+8)
10	100	10	16	1
11	110	11	16	1
12	120	12	16	1

Part B – Total Cost

① Sum = 1+1+3+1+5+1+1+1+9+1+1+1 = 26 = T_{12}

② Average Cost Per Call = $\frac{T_{12}}{12} = \frac{26}{12} = 2,17$

③. Formula for Total "Copy Work" for n pushBack Operations.

→ Resizing happens when the array becomes full (size reaches the capacity) at : 2, 4, 8, 16, ---

and when the capacity grows, all the excisting elements (2^t) must be copied to be new array.

The total copying work during n pushback operations forms a geometric series:

$\boxed{2+4+8+ \dots + 2^m}$, where $2^m < n \leq 2^{m+1}$

↳ $S = 2^{m+1} - 2$ → Since $2^{m+1} < 2n$, we get : $S < 2n$

The total copy cost is = $O(n)$

Amortized cost : The total work for n Pus Back operations includes : ∘ n normal insertions

∘ $O(n)$ total copying during resize

∘ $T(n) = O(n)$

∘ Amortized cost : $T(n)/n = O(1)$

Part C - Growth Factor Comparison

Suppose instead of doubling the capacity, pushBack grows by adding a fixed amount $k = 4$ (new capacity = old capacity + 4)

1. Redo the trace from part A (12 calls, starting from capacity 2).

Call #	Value	Size After	Cap After	Cost
1	10	1	2	1
2	20	2	2	1
3	30	3	6	3 (resize = 1+2)
4	40	4	6	1
5	50	5	6	1
6	60	6	6	1
7	70	7	10	7 (resize = 1+6)
8	80	8	10	1
9	90	9	10	1
10	100	10	10	1
11	110	11	14	11 (resize = 1+10)
12	120	12	14	1

2. Compare the total cost of both strategies.

T_{12} (part C) = $1+1+3+1+1+1+7+1+1+1+11+1 = 30$

- Doubling (Part A) = 26
- +4 (part C) = 30

untuk 12 call, +4 lebih mahal ($30 > 26$)

3. Derive $T(n)$ for both the doubling and the fixed-increment strategies. Show that doubling gives $O(n)$ total whereas fixed-increment gives $O(n^2)$ total. Express the amortised cost per call in each case.

- Doubling: Resize at 2, 4, 8, ... and totalcopy $< 2n$

  $T(n) = n + O(n) = O(n)$

  Amortized per call = $O(1)$.

- Fixed increment: Resize at 2, 6, 10, 14, ... and number of resizes $\approx n/4$

  → Total copy work: $2+6+10+14 \dots + (4m-2)$

  $S = \frac{m}{2}(2 + (4m-2)) = 2m^2$

  $T(n) = n + O(n^2) = O(n^2)$

  Amortized per call = $\frac{T(n)}{n} = \frac{O(n^2)}{n} = O(n)$.

- Doubling → $T(n) = O(n)$

  Amortized = $O(1)$

- Fixed Increment → $T(n) = O(n^2)$

  Amortized = $O(n)$

Doubling strategy is significantly more efficient. With doubling, the capacity grows exponentially, so the number of resizes is small.